

Paradigmas de Programación

TRABAJO PRÁCTICO N° 2

1. Cálculo Lambda

1.1. Teoría Lambda

Ejercicio 1.1. ¿Qué son las reglas de reducción? ¿Por qué se utilizan?

Ejercicio 1.2. ¿Qué significa la forma normal de una función lambda?

Ejercicio 1.3. ¿Toda función se puede expresar como un cálculo lambda? Justifique su respuesta.

Ejercicio 1.4. ¿Puede existir una λ -expresión bien formada que nunca llegue a forma normal?

Ejercicio 1.5. Explique qué ideas del λ -cálculo aparecen en lenguajes reales (por ejemplo: funciones anónimas, clausuras, alcance léxico). Nombre 2-3 y describa el vínculo.

Ejercicio 1.6. ¿Qué es una forma normal en λ -cálculo? ¿Qué diferencia conceptual hay entre “no puedo reducir más” y “no sé cómo reducir”?

Ejercicio 1.7. ¿Qué significa que dos expresiones sean α -equivalentes? ¿Es lo mismo que “son iguales”? ¿Qué cosas no se pueden cambiar con α -renombrado?

1.2. Práctica Lambda

Ejercicio 1.8. ¿Cuáles de las siguientes expresiones son λ -expresiones correctas según la gramática vista en clase? ¿Cuáles no lo son? Justifique cada respuesta.

A. $\lambda x.x$

D. λxx

G. $\lambda x.y$

B. $\lambda x.xy$

E. $\lambda x.xy\lambda x.xy$

H. $\lambda x.\lambda y.xy$

C. $\lambda x.x$

F. $(\lambda x. ((x y) . z))$

I. $(\lambda x. (x \lambda y. y))$

Ejercicio 1.9. Para cada inciso, haga explícitos los paréntesis de las siguientes λ -expresiones. Además, indique, para cada variable, cuáles de sus ocurrencias son libres y cuáles acotadas. Indique a qué abstracción- λ está ligada cada ocurrencia no libre.

A. $(\lambda x. x y) z$

B. $\lambda x.xz) \lambda y.w \lambda w.wyzx$

C. $\lambda x.xy \lambda x.yx$

D. $(\lambda x.\lambda y.xy) (\lambda z.z w)$

E. $\lambda x.(\lambda y.x (\lambda xyx)) x$

Ejercicio 1.10. Utilizando β -reducción, simplifique las siguientes expresiones hasta una forma final (normal), si existe. Si no existe, explique por qué.

- $((\lambda x.(x\ y))(\lambda z.z))$
- $((\lambda x.((\lambda y.(x\ y))x))(\lambda z.w))$
- $((((\lambda f.((\lambda g.(\lambda x.((f\ x)\ (g\ x))))(\lambda m.(\lambda n.(n\ m)))))(\lambda n.z))\ p)$
- $((\lambda x.(x\ x))(\lambda x.(x\ x)))$
- $((\lambda f.((\lambda g.((f\ f)g))(\lambda h.(k\ h))))(\lambda x.(\lambda y.y)))$
- $(\lambda g.((\lambda f.((\lambda x.(f\ (x\ x)))(\lambda x.(f\ (x\ x))))\ g))$

Ejercicio 1.11. Para los siguientes términos, marque la opción correcta:

- $\lambda f. (\lambda x. f (\lambda y. x\ x\ y)) (\lambda x. f (\lambda y. x\ x\ y))) (\lambda g. \lambda y. y) (\lambda h.h)$
 - $(\lambda f. (\lambda x. f (\lambda y. x\ x\ y)) (\lambda x. f (\lambda y. x\ x\ y))) (\lambda g. \lambda y. y)$
 - $\lambda h.h$
 - $(\lambda x. (\lambda g.\lambda y.y) (\lambda y. x\ x\ y)) (\lambda x. (\lambda g. \lambda y. y) (\lambda y. x\ x\ y))$
 - $\lambda x. (\lambda g.\lambda y.y) (\lambda y. x\ x\ y)$
 - NINGUNO
- $(\lambda f. (\lambda x. f (\lambda y. x\ x\ y)) (\lambda x. f (\lambda y. x\ x\ y))) (\lambda g. g) (\lambda h.h)$
 - $(\lambda f. (\lambda x. f (\lambda y. x\ x\ y)) (\lambda x. f (\lambda y. x\ x\ y))) (\lambda g. g)$
 - $\lambda h.h$
 - $(\lambda x. (\lambda g.g) (\lambda y. x\ x\ y)) (\lambda x. (\lambda g.g) (\lambda y. x\ x\ y))$
 - $\lambda x. (\lambda g.g) (\lambda y. x\ x\ y)$
 - NINGUNO

2. Programación Funcional - Teoría

Ejercicio 2.1. ¿Qué es la programación funcional y qué características presenta?

Ejercicio 2.2. ¿Qué características presenta un programa escrito en un lenguaje funcional con respecto a un programa escrito en un lenguaje imperativo? ¿Qué rol cumple la computadora?

Ejercicio 2.3. ¿Cuál es el concepto de variables en los lenguajes funcionales?

Ejercicio 2.4. Enumere y explique brevemente cuáles son las diferencias entre un lenguaje funcional, un lenguaje lógico y un lenguaje procedimental.

Ejercicio 2.5. ¿En qué situaciones cree que sería mejor utilizar el paradigma funcional?

Ejercicio 2.6. ¿El paradigma funcional se basa en el funcionamiento de la máquina de Von Neumann?

Ejercicio 2.7. Explique qué es el Cálculo Lambda y por qué los lenguajes funcionales se basan en él.

Ejercicio 2.8. Explique qué es la transparencia referencial.

Ejercicio 2.9. Haskell es un lenguaje de programación funcional con un sistema de tipos (solo existe una respuesta correcta). Indique cuál(es) de las siguientes afirmaciones es/son verdadera(s):

- a) Cada expresión debe tener un tipo, que se calcula antes de evaluar la expresión mediante un proceso llamado inferencia de tipos.
- b) La inferencia de tipos precede siempre a la evaluación.
- c) Los errores de tipos no ocurren durante la evaluación, pero sí pueden darse en la inferencia de tipos.
- d) Todas son verdaderas.

Ejercicio 2.10. En Haskell, las listas deben ser homogéneas, mientras que las tuplas pueden ser heterogéneas. VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.11. La aridad de una tupla es el número de componentes. Las tuplas pueden tener aridad 0, aridad 2 y aridad n , pero no pueden tener aridad 1. VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.12. Una tupla con aridad 2 tiene un tipo diferente a una tupla con aridad 3 (aunque contengan el mismo tipo de valores). VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.13. Las listas pueden ser infinitas, mientras que las tuplas deben tener una aridad finita para garantizar que los tipos de tuplas siempre puedan calcularse antes de la evaluación. VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.14. Un tipo polimórfico es un tipo que contiene una o más variables de tipo, y una función que tiene un tipo polimórfico se llama función polimórfica. VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.15. En Haskell, las variables de tipo se escriben usualmente en minúscula (por ejemplo, `a`, `b`). Explique qué significa que una función tenga una variable de tipo en su firma (por ejemplo, `id :: a -> a`) y por qué eso indica que la función es polimórfica.

Ejercicio 2.16. Explique qué es la parcialización y la curificación, y cómo se relacionan entre sí.

Ejercicio 2.17. Defina el proceso de evaluación perezosa en un programa funcional. ¿Qué ventajas ofrece sobre una evaluación ansiosa?

Ejercicio 2.18. ¿Qué significa que una función sea de orden superior? Justifique su respuesta.

Ejercicio 2.19. Las mónadas nos permiten resolver diferentes problemas. ¿Cuáles serían? Justifique cómo estos problemas afectan a un lenguaje funcional.

Ejercicio 2.20. Defina y describa, mediante ejemplos, los constructores de tipo versus los constructores de datos utilizados en Haskell. Incluya ejemplos con `data`.

3. Programación Funcional - Practica

Ejercicio 3.1. Calcular la media aritmética de tres valores numéricos.

Ejercicio 3.2. Calcular el máximo entre 3 números (puede utilizar la función predefinida `max`).

Ejercicio 3.3. Definir una función tal que reciba un par (x, y) y retorne el cuadrante. El par no está sobre los ejes x ni y .

Ejercicio 3.4. Definir la función `igualesTres` que verifica que tres elementos x , y , z son iguales.

Ejercicio 3.5. Definir la función `diferentesTres` que verifica que tres elementos x , y , z son diferentes.

Ejercicio 3.6. Definir la función `esPar` que verifica si un número es par.

Ejercicio 3.7. Definir la función `esPrimo` que verifica si un número es primo.

Ejercicio 3.8. Definir la función `esBisiesto` que verifica si un año es bisiesto.

3.1. Uso de guardas, patrones y tipos definidos por el usuario

Ejercicio 3.9. Definir las siguientes funciones utilizando guardas, patrones y datos definidos por el usuario. En ningún caso, utilizar funciones predefinidas de Haskell.

A. Calcular la negación, la conjunción y la disyunción usando patrones con valores.

B. Determinar si un número es par.

C. Con base en el puntaje obtenido en un examen (0–100), se desea saber la condición del alumno (aprobado o desaprobado).

D. Mezcla de colores: dado el porcentaje de dos colores primarios, obtener un color secundario (y sucesivos).

E. Clasificar personas por edad:

- Menor: < 13
- Adolescente: 13 a 17
- Adulto: 18 a 69
- Anciano: > 69

F. Clasificar personas (18 o más) por nivel de estudios: primarios, secundarios o superiores.

G. Clasificar triángulos por lados: equilátero, isósceles, escaleno o “no es triángulo” (desigualdad triangular).

H. Definir el tipo Figura (círculo, rectángulo, cuadrado) y `area :: Figura -> Float`.

3.2. Listas

Ejercicio 3.10. Definir la función `divisores` tal que `divisores x` es la lista de los divisores de `x`.

```
1 divisores 4 == [1,-1,2,-2,4,-4]
2 divisores (-6) == [1,-1,2,-2,3,-3,6,-6]
```

Ejercicio 3.11. Dada una cantidad de segundos, devolver la cantidad equivalente de horas, minutos y segundos.

Ejercicio 3.12. Definir una función que devuelva una lista de números primos hasta un valor `N`.

Ejercicio 3.13. Definir la función que convierte un número decimal en su correspondiente número binario. Los binarios deben almacenarse como una lista.

Ejercicio 3.14. Crear una función `ocurrencias` que tome un elemento y una lista, y devuelva el número de ocurrencias del elemento en la lista.

Ejercicio 3.15. Crear la función `filtrarPositivos` que, dada una lista de enteros, devuelva una lista con los elementos positivos.

Ejercicio 3.16. Crear la función `reversa` que, dada una lista de enteros, devuelva la lista con los mismos elementos de atrás para adelante.

Ejercicio 3.17. Definir la función `sumaPar` que, dada una lista de pares, devuelva una nueva lista en la que cada elemento sea la suma de los elementos de cada par.

Ejercicio 3.18. Definir una función `zipSort` que, dadas dos listas de enteros de igual longitud, devuelva una lista de pares `(min, max)`, donde `min` y `max` son el mínimo y el máximo entre los elementos de ambas listas en la misma posición.

3.3. Por comprensión y recursión (sin funciones de orden superior ni funciones predefinidas)

Ejercicio 3.19. Definir las siguientes funciones:

- A. `sumaElementos`: suma todos los elementos de una lista de números.
- B. `soloPares`: devuelve la lista de números pares hasta el valor `n`.
- C. `todosVerdaderos`: devuelve `True` si todos los booleanos de una lista son `True`.
- D. `incrementar`: incrementa todos los elementos de una lista de enteros.
- E. `cuadrados`: devuelve la lista de cuadrados de una lista dada.

F. divisores: lista de divisores del número ingresado.

G. sumaParesDesde: dados n y m, generar pares hasta n, pero solo los mayores que m.

H. sumaDeCuadrados:

```
1 ghci> sumaDeCuadrados 4
2 30
```

I. sumaDeCuadradosPares (según enunciado):

```
1 ghci> sumaDeCuadradosPares 4
2 84
```

J. productoEscalar:

```
1 ghci> productoEscalar [1,3,5] [2,4,6]
2 44
```

K. masDe: devuelve las sublistas con longitud mayor que n.

L. posicionEnLista: posición inicial de una sublista en una lista:

```
1 ghci> posicionEnLista [1] [4,5,1,2,5,1]
2 2
3 ghci> posicionEnLista [7] [4,5,1,2,5,1]
4 Nothing
```

M. buscaCrucigrama:

```
1 buscaCrucigrama :: Char -> Int -> Int -> [String] -> [String]
2
3 ghci> buscaCrucigrama 'c' 1 7 ["ocaso", "acabado", "ocupado"]
4 ["acabado", "ocupado"]
```

3.4. Inferencia de tipos

Ejercicio 3.20. Inferir, de ser posible, los tipos de las siguientes funciones, y explicar cómo trabaja cada una y cómo se infieren los tipos.

```
1 suma x y = x + y
2
3 impar = not . Even
4
5 modulus = sqrt . sum . map square
6
7 factorial 0 = 1
8 factorial n = n * factorial (n - 1)
9
10 dosVeces f x = f (f x)
11
12 incLista [] = []
13 incLista (x:xs) = (x + 1) : incLista xs
14
15 dosVecesL xs = map (*2) xs
16
17 sumaBis xs ys = zipWith (+) xs ys
18
19 fc = \a -> (\b -> 2 * a + b)
20
21 f x m z = (head m) > (map (\n -> n x) z)
```

3.5. Funciones predefinidas

Ejercicio 3.21. Teniendo en cuenta la definición de la función predefinida `foldr`:

```
1 ghci> :t foldr
2 foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
```

Redefinir (sin usar las predefinidas equivalentes):

- A. la suma de una lista de enteros
- B. el producto de una lista de enteros
- C. la longitud de una lista

Ejercicio 3.22. Teniendo en cuenta las siguientes funciones predefinidas en Haskell:

```
1 map :: (a -> b) -> [a] -> [b]
2 filter :: (a -> Bool) -> [a] -> [a]
3 takeWhile :: (a -> Bool) -> [a] -> [a]
4 dropWhile :: (a -> Bool) -> [a] -> [a]
5 iterate :: (a -> a) -> a -> [a]
6 zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

Definir las siguientes funciones de alto orden:

- A. Función que toma una lista y otra función, aplica la función a cada elemento y devuelve la nueva lista.
- B. Filtrar los números de una lista que sean menores que 5.
- C. Función que multiplique todos los elementos mayores que 0.
- D. `concatenaLL yss`: concatena una lista de listas.
- E. `filtraUno f cond xs`: aplica `f` a los elementos de `xs` que cumplen `cond`.

Incluya ejemplos de uso en `ghci`.

4. Evaluación perezosa y ansiosa

Ejercicio 4.1. Evaluar el funcionamiento mediante evaluación ansiosa y perezosa:

```
1 a) inf :: Int
2     inf = 1 + inf
3
4 b) cuadrado :: Int -> Int
5     cuadrado n = n * n
6
7 c) square x = x * x
8     square (square (square 2))
9
10 d) naturalesDesde x = x : naturalesDesde (x+1)
11
12 e) mult(a, b) = a * a
13
14 f) foldl :: (b -> a -> b) -> b -> [a] -> b
15     foldl _ z [] = z
16     foldl f z (x:xs) = foldl f (f z x) xs
17
18     foldl (+) 0 [1,2,3]
```

4.1. Ejercicios integradores

Ejercicio 4.2 (Edad en planetas). Si dijéramos que alguien tiene una edad de 1000000000 segundos, también podríamos decir que tiene 31.69 años terrestres. El periodo orbital equivale a:

- Mercurio: 0.2408467 años terrestres
- Venus: 0.61519726 años terrestres
- Tierra: 1.0 años terrestres (365.25 días) o 31557600 segundos
- Marte: 1.8808158 años terrestres
- Júpiter: 11.862615 años terrestres
- Saturno: 29.447498 años terrestres
- Urano: 84.016846 años terrestres
- Neptuno: 164.79132 años terrestres

Calcular, en base a una cantidad de segundos y un planeta, los años equivalentes:

```
1 ghci> edad Marte 1000000000
2 16.848055
```

Ejercicio 4.3 (R2-D2 super powa). Ayudar a evaluar el funcionamiento de un lindo R2-D2. Se necesita hacer una prueba de fábrica de robots necesita un programa para verificar los movimientos del robot.

Los robots tienen tres movimientos posibles:

- doblar a la derecha
- doblar a la izquierda
- avanzar

Los robots se colocan en una cuadrícula hipotéticamente infinita donde el inicio de los ejes es en la esquina superior oeste, con una dirección particular (Norte, Este, Sur u Oeste) en un conjunto de coordenadas (x, y), por ejemplo, (3,8), con coordenadas que aumentan hacia el norte y el este.

Luego, el robot recibe una serie de instrucciones, momento en el cual la prueba verifica la nueva posición del robot y en qué dirección apunta.

Una cadena de letras "DAALAL" significa:

- Girar a la derecha
- Avanzar dos veces
- Girar a la izquierda
- Avanzar una vez
- Gira a la izquierda una vez más

Un robot comienza en (7, 3) ubicado hacia el norte. Luego, ejecutando esta secuencia de instrucciones queda en (9, 4) hacia el oeste.

Se pide modelar un robot que se mueva desde una posición (x,y) a otro par de coordenadas mediante un conjunto de instrucciones.

Recomendaciones:

- Se aconseja crear el tipo de datos Robot y ubicación e implementar las siguientes funciones:
 - crearRobot
 - simular
 - coordenadas

- girar a la izquierda
- girar a la derecha

NOTA: Las funciones a definir son de ayuda, si ud. desea implementar más o menos funciones puede hacerlo tranquilamente.

Ejercicio 4.4 (Pinky y Cerebro). Modelar animales de laboratorio:

- Un animal tiene: coeficiente intelectual (número), especie (String) y capacidades ([String]).
- Transformaciones:
 - `inteligenciaSuperior`: incrementa en `n` su coeficiente.
 - `pinkificar`: quita todas las habilidades.
 - `superpoderes`: agrega habilidades nuevas. Si es elefante: agrega "no tenerle miedo a los ratones". Si es ratón con `CI > 100`: agrega "hablar"; si no, queda igual.
- Criterios:
 - `antropomorfico`: tiene "hablar" y `CI > 60`.
 - `noTanCuerto`: tiene más de 2 habilidades `pinkiescas`. (Asumir `pinkiesco :: String -> Bool`.)
- Experimentos: un experimento tiene transformaciones y un criterio de éxito. Definir `experimentoExitoso` y dar un ejemplo de consulta para el caso pedido en el enunciado.

Ejercicio 4.5. Un centro de alertas climáticas recibe mediciones de estaciones (temperatura, humedad y viento). Se quiere un programa funcional en Haskell que limpie datos, combine reglas y genere un reporte simple de alertas por estación/zona. No se permite usar `error` ni excepciones: todo debe modelarse con tipos y funciones.

A. Definir tipos para:

- **Zona** (por ejemplo, un String o newtype).
- **Estación** con id, nombre y zona.
- **Medición** con estación, timestamp simple (un Int o newtype), temperatura, humedad y viento.
- **Severidad** con cuatro niveles (por ejemplo: `SinAlerta | Preventiva | Alerta | Emergencia`).
- **Motivo** como tipo algebraico (por ejemplo: `CalorExtremo, VientoFuerte, DatosSospechosos String`).
- Un tipo para el resultado de evaluar una medición: `severidad + lista de motivos`.

B. Cargar (o proponer) 8–10 mediciones de ejemplo que incluyan: mediciones duplicadas (misma estación y timestamp), humedad fuera de 0–100, viento negativo, temperatura extrema y al menos dos zonas distintas.

C. Implementar una función que, dada una medición, devuelva un resultado de “validación” (sin eliminar la medición), marcando como `Preventiva` si hay datos sospechosos y agregando motivos por cada problema detectado. Rango sugerido (configurable): temperatura entre -10 y 55, humedad 0–100, viento ≥ 0 . La idea es que una medición puede ser “válida” (`SinAlerta` por datos) o “sospechosa” (`Preventiva` por datos), pero siempre se conserva.

D. Luego, implementar una función que elimine duplicados por clave (`idEstacion, timestamp`) quedándose con la medición “más confiable”: definir y justificar un criterio (por ejemplo, la que tenga menos motivos de datos sospechosos; si empatan, quedarse con la primera o la última).

E. Definir el tipo `Regla` como una función que recibe una `Medicion` y devuelve un resultado (`severidad + motivos`). Implementar al menos tres reglas:

- Regla de calor: sube la severidad según temperatura y humedad (por ejemplo: si `temp ≥ 40` y `hum ≥ 40` $\Rightarrow$ `Emergencia`; si `temp ≥ 35` y `hum ≥ 35` $\Rightarrow$ `Alerta`; si `temp ≥ 33` $\Rightarrow$ `Preventiva`; si no $\Rightarrow$ `SinAlerta`). Los umbrales pueden variar, pero deben quedar claros.

- Regla de viento: sube la severidad según viento (por ejemplo: ≥ 90 Emergencia, ≥ 70 Alerta, ≥ 50 Preventiva).
- Regla de datos: usa la validación del punto (3) para aportar Preventiva y motivos si hay inconsistencias.

F. Ahora, definir un combinador `combinarReglas :: [Regla] -> Regla` que aplique todas las reglas a una medición y produzca un único resultado final, eligiendo la severidad más alta y juntando motivos sin repetir. Justificar el criterio para “peor severidad” y cómo se evitan duplicados en los motivos.

G. Definir `generarReporte :: [Medicion] -> Reporte` (se puede definir `Reporte` como una lista de ítems) que:

- deduplique las mediciones,
- evalúe el motor de reglas en cada medición,
- agrupe por zona (o, al menos, incluya la zona en cada ítem),
- ordene el resultado por severidad descendente (Emergencia primero) y, si empatan, por id de estación,
- incluya, para cada estación: severidad final, motivos y cuántas mediciones “sospechosas” tuvo esa estación (según la validación del punto 3) dentro del conjunto deduplicado.

Ejercicio 4.6 (Academia de Magia). En la academia de magia *Arcana Major*, distintos magos realizan hechizos que requieren ciertos conocimientos, energía mágica y, en algunos casos, artefactos especiales.

Cada mago posee un nivel de energía, una lista de habilidades mágicas y un inventario de artefactos. Por otro lado, cada hechizo tiene un nivel de dificultad, una cantidad de maná requerido, una habilidad necesaria y opcionalmente un artefacto específico.

Se desea modelar este dominio utilizando Haskell para analizar qué magos pueden ejecutar determinados hechizos y calcular recompensas mágicas acumuladas.

Se pide:

A. Definir los siguientes tipos de datos utilizando `data`:

- Mago: nombre, nivel de energía, habilidades (lista de strings), inventario (lista de strings)
- Hechizo: nombre, dificultad, recompensa, habilidad requerida, artefacto requerido (usar `Maybe`)
- Academia: lista de magos y lista de hechizos

B. Crear al menos 4 magos y 4 hechizos de ejemplo.

C. Implementar una función:

```
buscarMago :: String -> Academia -> Maybe Mago
```

que devuelva un mago dado su nombre, o `Nothing` si no existe.

D. Implementar:

```
puedeLanzar :: Mago -> Hechizo -> Bool
```

Un mago puede lanzar un hechizo si:

- su energía es suficiente para la dificultad
- posee la habilidad requerida
- si el hechizo requiere artefacto, debe tenerlo en su inventario

E. Implementar:

```
magosAptos :: Hechizo -> Academia -> [String]
```

que devuelva los nombres de los magos que pueden lanzar un hechizo.

Restricción: debe resolverse utilizando funciones de orden superior, composición y aplicación parcial.

F. Implementar:

```
recompensaTotal :: Mago ->[Hechizo] ->Int
```

que calcule la suma de recompensas de los hechizos que el mago puede realizar.

Restricción: no utilizar funciones predefinidas de listas como map, filter, foldr, etc.

G. Explicar:

- dónde se utiliza Maybe y por qué
- ejemplos de aplicación parcial
- uso de composición
- diferencia entre uso de recursión y funciones de orden superior

Ejercicio 4.7 (Sistema de Exploración Espacial). Una organización interplanetaria gestiona una flota de naves dedicadas a la exploración de distintos entornos espaciales. Cada nave posee ciertas características, capacidades operativas y recursos disponibles que determinan su desempeño en diferentes escenarios.

Por otro lado, las misiones presentan distintos niveles de complejidad y pueden requerir ciertas condiciones específicas para ser llevadas a cabo. Algunas de estas condiciones pueden ser obligatorias, mientras que otras pueden no estar presentes en todos los casos.

Se desea construir un modelo en Haskell que permita representar este sistema y analizar la relación entre las naves y las misiones disponibles.

Se pide:

A. Proponer una representación adecuada para modelar:

- las naves
- las misiones
- el sistema general que las agrupa

Nota: Justificar brevemente las decisiones de modelado tomadas, especialmente en aquellos casos donde cierta información pueda o no estar presente.

B. Definir una función que permita recuperar información sobre una nave a partir de algún identificador.

C. Definir una función que determine si una nave está en condiciones de realizar una misión.

A tener en cuenta:

- pueden existir múltiples criterios de validación
- algunas condiciones pueden depender de información opcional
- la decisión final debe ser un valor booleano

D. Construir una función que permita obtener, a partir de una misión, el conjunto de naves que podrían llevarla a cabo.

Restricción:

- se espera el uso de funciones de orden superior
- se recomienda el uso de composición de funciones
- considerar la reutilización de funciones previamente definidas

E. Diseñar una función que permita analizar el rendimiento potencial de una nave considerando un conjunto de misiones disponibles.

F. (Opcional) Definir una función que permita seleccionar una misión “óptima” para una nave según algún criterio.